



ASIGNATURA: Algoritmos y Estructuras de Datos
AÑO: 6to CICLO LECTIVO: 2025
DOCENTE: Romero Fabrizio Nahuel

Trabajo Práctico Interdisciplinario

Asignatura: Algoritmos y estructuras de datos

Objetivo: Evaluar la comprensión teórica y la capacidad práctica del alumno en desarrollar algoritmos en Python que faciliten la lectura/escritura en base de datos relacionales.

Parte I: Cuestionario Teórico

1. Fundamentación Teórica (Orientada a la Práctica)

Antes de comenzar la implementación, investigue y desarrolle los siguientes conceptos teóricos, explicando cómo se relacionan directamente con las tareas que realizará en este trabajo práctico:

1. Estructuras de Datos en Pandas:

- **DataFrame vs. Array Bidimensional:** Compare un DataFrame de Pandas con una estructura de datos clásica como un array bidimensional (matriz) o una lista de listas. Discuta las ventajas del DataFrame (indexación basada en etiquetas, manejo de tipos de datos heterogéneos, operaciones vectorizadas).
- **Series vs. Array Unidimensional:** Explique qué es un objeto **Series** en Pandas y cómo se basa en los **ndarray** de NumPy.

2. Algoritmos de Procesamiento (Vectorización):

- **Vectorización (NumPy/Pandas) vs. Iteración (Bucles **for**):** Explique el concepto de vectorización. ¿Por qué calcular el puntaje promedio usando `df['puntaje'].mean()` es algorítmicamente más eficiente (en términos de velocidad de ejecución en Python) que iterar sobre cada fila con un bucle **for** y sumar los valores manualmente? Relaciónelo con la implementación subyacente de NumPy (operaciones en C).

3. Algoritmos de Ordenamiento y Búsqueda en el Análisis:

- **Eficiencia en la Búsqueda:** Cuando utiliza una expresión como `df[df['puntaje'] > 90]`, ¿qué tipo de operación de búsqueda o filtrado se está realizando?
- **Ordenamiento:** Cuando se pide encontrar los "mejores puntajes", se utiliza una función como `df.sort_values('puntaje')`. Investigue qué algoritmos de ordenamiento (ej. Quicksort, Mergesort, Timsort) suelen utilizar las bibliotecas modernas como Pandas y por qué son eficientes $O(n \log n)$

Parte II: Actividades Prácticas

Desarrollo Práctico: Análisis del Torneo

Deberá escribir un programa en Python (se recomienda usar un J para facilitar la exploración y visualización) que realice los siguientes pasos.

Paso 1: Configuración del Entorno

Asegúrese de tener instaladas las bibliotecas necesarias:

Bash

- pip install pandas numpy matplotlib mysql-connector-python

Paso 2: Ingesta de Datos (Elegir una opción)

El alumno debe implementar **ambos métodos** de conexión, aunque puede elegir uno para realizar el análisis principal.

- **Opción A (Fuente CSV):**

- Cargar los datos desde el archivo `torneo_quizizz.csv` en un DataFrame de Pandas. ○ `df = pd.read_csv('torneo_quizizz.csv')`

- **Opción B (Fuente MySQL):**

- Conectarse a la base de datos creada por la otra asignatura (que contiene los mismos datos del CSV).
- Utilizar `mysql-connector-python` para establecer la conexión y ejecutar una consulta `SELECT * FROM tabla_torneo`.

- Cargar los resultados de la consulta directamente en un DataFrame de Pandas. TP Interdisciplinario

Ciclo lectivo: 2025

Fragmento de código de ejemplo (Opción B):

Python

- `import mysql.connector`
- `import pandas as pd`

```
cnx = mysql.connector.connect(user='...', password='...',
```

```
host='127.0.0.1', database='db_torneo')

query = "SELECT * FROM resultados_alumnos"

df = pd.read_sql(query, cnx)

cnx.close()
```

Paso 3: Limpieza y Preparación de Datos (EDA)

Los datos de Quizizz (o cualquier CSV) rara vez vienen listos para analizar. Esta es una etapa crítica.

- **Inspección:** Revise los tipos de datos (`df.info()`). Es común que los puntajes o tiempos se lean como texto (`object`).
- **Conversión de Tipos:**
 - Convierta las columnas de puntaje a numérico (`pd.to_numeric`).
 - Convierta las columnas de fecha (si existen) a formato `datetime` (`pd.to_datetime`).
- **Manejo de Tiempos:** El "tiempo de respuesta" puede estar en formato "1m 30s" o "45s". Cree una función para normalizar este tiempo a una sola unidad (ej. **segundos totales**) para poder compararlo.
- **Manejo de Nulos:** Verifique si hay datos faltantes (`df.isnull().sum()`) y decida una estrategia (ej. eliminar la fila, rellenar con cero o la media).

TP Interdisciplinario Ciclo lectivo: 2025

Paso 4: Análisis Estadístico y Generación de Gráficos

Genere las siguientes estadísticas y visualizaciones. Cada estadística debe ir acompañada de un gráfico (usando `matplotlib`) que la ilustre claramente.

1. Top 10 - Mejores Puntajes:

- **Estadística:** Un DataFrame que muestre a los 10 alumnos con los puntajes más altos.
- **Gráfico:** Un gráfico de barras horizontales (`plt.barh()`) que muestre a estos 10 alumnos y sus puntajes.

2. Top 10 - Alumnos Más Veloces (Mejor Tiempo de Respuesta):

- **Estadística:** Un DataFrame con los 10 alumnos que tuvieron el mejor tiempo de respuesta promedio (o total, según se defina) en segundos.
- **Gráfico:** Un gráfico de barras similar al anterior, mostrando el tiempo en segundos.

3. Ganadores por Fechas:

- **Estadística:** Si el torneo se jugó en varias fechas, determine quién fue el alumno con el puntaje más alto en *cada* fecha. (Pista: `df.groupby('Fecha')['Puntaje'].idxmax()`).
- **Gráfico:** Un gráfico de líneas que muestre la evolución del "puntaje ganador" a lo largo de las fechas del torneo.

4. Distribución de Puntajes:

- **Estadística:** Calcular la media, mediana, desviación estándar, puntaje mínimo y máximo de todo el torneo (`df['Puntaje'].describe()`).
- **Gráfico:** Un **histograma** (`plt.hist()`) que muestre cuántos alumnos cayeron en diferentes rangos de puntaje (ej. 0-10, 10-20, ...).

5. Automatización:

- Cree una función que reciba el DataFrame como parámetro y genere automáticamente un informe (ej. un archivo PDF o un conjunto de imágenes `.png`) con todos los gráficos y estadísticas guardados.

3. Entregables

Se deberá entregar en una carpeta comprimida:

1. **Informe Teórico:** Un documento (PDF) que responda a la sección "1. Fundamentación Teórica".
2. **Código Fuente:** El archivo `.py` o el Jupyter Notebook (`.ipynb`) con todo el proceso: conexión (ambos métodos), limpieza, análisis y generación de gráficos. El código debe estar debidamente comentado.
3. **Gráficos Generados:** Las imágenes (ej. `.png`) de los 4 gráficos solicitados.

Respuestas

Teoría:

1. Fundamentación Teórica (Orientada a la Práctica)

Estructuras de Datos en Pandas

Un DataFrame en Pandas es una estructura de datos bidimensional que se asemeja a una tabla, compuesta por filas y columnas, donde cada columna puede tener un nombre (etiqueta) y un tipo de dato diferente. Por ejemplo, una columna puede contener números enteros, otra texto y otra valores flotantes. Esto lo diferencia de un array bidimensional clásico, como una lista de listas en Python o un arreglo de NumPy, donde todos los elementos suelen tener el mismo tipo de dato y se accede a ellos únicamente mediante índices numéricos.

El DataFrame ofrece ventajas importantes: permite la indexación basada en etiquetas, lo que significa que se puede acceder a los datos por el nombre de la columna o del índice, no solo por posición. Además, admite distintos tipos de datos en una misma estructura, y permite realizar operaciones vectorizadas, es decir, aplicar cálculos sobre columnas enteras sin usar bucles. Estas características hacen que el trabajo con datos sea mucho más rápido, legible y eficiente, especialmente cuando se procesan grandes volúmenes de información. Por ejemplo, calcular promedios, ordenar filas o filtrar registros se logra con una sola línea de código.

En cambio, un array bidimensional (o una lista de listas) no tiene etiquetas ni funciones integradas para análisis de datos. En ese caso, cualquier operación, como calcular el promedio de una columna, requeriría recorrer manualmente el arreglo con bucles y escribir más código, lo que aumenta el tiempo de desarrollo y la posibilidad de errores.

Dentro de Pandas también existe el objeto Series, que representa una estructura unidimensional con etiquetas. Puede verse como una columna individual de un DataFrame o como una lista con nombres asociados a cada valor. Las Series están basadas en los arreglos `ndarray` de NumPy, por lo tanto heredan su eficiencia en el manejo de datos numéricos, pero

añaden la posibilidad de tener índices personalizados. Gracias a esto, se pueden realizar operaciones matemáticas o de filtrado usando las etiquetas en lugar de posiciones. Por ejemplo, una Series puede almacenar los puntajes de los estudiantes y permitir acceder directamente al puntaje de “Ana” sin necesidad de saber en qué posición está.

Esta relación entre Series y DataFrame es fundamental: un DataFrame está compuesto por varias Series que comparten el mismo índice. En la práctica, esto permite manejar los datos de manera estructurada y semántica, lo que simplifica mucho las tareas de análisis.

2. Algoritmos de Procesamiento (Vectorización)

La vectorización es un concepto central en el uso de Pandas y NumPy. Consiste en aplicar operaciones matemáticas o lógicas sobre conjuntos completos de datos a la vez, sin necesidad de usar bucles explícitos en Python. Cuando escribimos una instrucción como `df['puntaje'].mean()`, Pandas no recorre fila por fila con un “for”, sino que delega la operación a funciones escritas en lenguaje C optimizado, que trabajan directamente sobre los arreglos de memoria del tipo `ndarray`. Esto hace que el cálculo sea muchísimo más rápido y eficiente.

En cambio, si intentáramos calcular el promedio de puntajes usando un bucle `for`, sumando los valores uno por uno y dividiendo al final, el intérprete de Python tendría que ejecutar cada iteración de manera secuencial, lo que es mucho más lento. Python no está diseñado para procesar millones de operaciones numéricas rápidamente, mientras que NumPy y Pandas sí lo

están, gracias a su implementación en C y al uso de operaciones vectorizadas.

Por lo tanto, la vectorización no solo simplifica el código (porque evita escribir bucles manuales), sino que también mejora el rendimiento. En el contexto de un trabajo práctico de análisis de datos, usar funciones como `mean()`, `sum()`, `max()` o `apply()` sobre un DataFrame permite obtener resultados de forma mucho más rápida y clara que hacerlo mediante iteraciones tradicionales.

3. Algoritmos de Ordenamiento y Búsqueda en el Análisis

Cuando se usa una expresión como `df[df['puntaje'] > 90]` en Pandas, lo que se está realizando es una operación de filtrado vectorizado. Internamente, Pandas evalúa la condición `df['puntaje'] > 90` y genera una Serie booleana, donde cada valor indica si la fila cumple o no la condición. Luego, esta Serie se utiliza para seleccionar solo las filas correspondientes en el DataFrame original. Es decir, no se trata de una búsqueda secuencial como la que haría un bucle, sino de una comparación masiva en paralelo, optimizada a nivel de memoria, gracias a la implementación en NumPy.

Por otro lado, cuando se ordenan los datos con una instrucción como `df.sort_values('puntaje')`, Pandas aplica algoritmos de ordenamiento eficientes. Normalmente, utiliza implementaciones modernas de algoritmos como **Quicksort**, **Mergesort** o **Timsort** (dependiendo del tipo de datos y del

parámetro elegido). Estos algoritmos tienen una complejidad promedio de $O(n \log n)$, lo que significa que pueden ordenar grandes volúmenes de datos de forma rápida y estable. Por ejemplo, Timsort, que es el algoritmo que usa Python por defecto, combina las ventajas de Mergesort y de Insertion Sort, siendo muy eficiente con datos parcialmente ordenados, algo muy común en conjuntos de datos reales.

En la práctica, esto permite ordenar un DataFrame completo por cualquier columna con una sola línea de código, sin tener que programar manualmente el algoritmo de ordenamiento. Gracias a estas optimizaciones, Pandas facilita enormemente tareas como encontrar los mejores puntajes, ordenar listas de estudiantes o realizar búsquedas filtradas de manera veloz y confiable.

